

ISM3232 — Week 4 Lab

Search Tools, the Submission Ritual & Git — Instructor Facilitation Guide

ISM3232 · Business Application Development · USF Muma College of Business

Module 04 · Unit 1 · Developer Foundations

Contents

1 Session Snapshot	2
2 Learning Objectives	2
3 Before Class — Setup Checklist	3
4 Materials Needed	3
5 Timing Plan (75 minutes)	3
6 Segment-by-Segment Walkthrough	4
6.1 Intro (0:00–0:04)	4
6.2 Part 1 — Search Tools (0:04–0:16, 12 min)	4
6.3 Part 2 — Write a Test File (0:16–0:28, 12 min)	7
6.4 Part 3 — Pre-Submission Ritual (0:28–0:46, 18 min)	10
6.5 Part 4 — First GitHub Push (0:46–1:04, 18 min)	13
6.6 Stretch — fzf or zoxide (1:04–1:10, as time allows)	15
7 Wrap-Up (last ~5 minutes)	16
8 Appendix A — Full Command Reference	16
9 Appendix B — Extra Practice (only if the class finishes early)	17

1 Session Snapshot

Course	ISM3232 — Business Application Development
Session	Week 4 Lab — Search Tools, the Submission Ritual & Git
Unit	Unit 1 · Developer Foundations
Class length	Full class period (75 minutes)
Format	Live terminal code-along
Prerequisites	Weeks 1–3: verified environment, <code>ism3232/</code> structure, navigation, <code>venv + .gitignore + .zshrc</code> aliases (marked Midterm-Eligible)
Student-facing lab page	Week 4 In-Class Lab — Module 2E–2H & Module 3, “Search, Ritual, and Git”
Parts covered	Part 1 (search tools) – Part 4 (first GitHub push) + Stretch (<code>fzf/zoxide</code>)
Submission	3 screenshots + <code>tests/test_week4.py</code> , GitHub repo URL, Canvas, completion credit

The lab page’s own warning is worth reading to the class verbatim: **this is the most important lab of Unit 1**. Everything from here forward — every remaining module’s submission — runs through the exact five-step ritual practiced today: format, lint, test, then stage/commit/push. Instructors check the Git log, and a missing `ruff` or `pytest` step is visible there, not hideable. Protect real, unhurried time for Part 3’s ritual — it’s short to type but is meant to become the single most repeated sequence of commands in the entire course, and it deserves to be executed carefully today, once, correctly, rather than rushed through as a formality.

2 Learning Objectives

By the end of this class period, students should be able to:

1. Search a codebase’s contents with `ripgrep (rg)`, locate files by name/pattern with `find`, and visualize structure with `tree`.
2. Write and run a `pytest` test file, and read `pytest -v`’s pass/fail output.
3. Execute the complete pre-submission ritual — confirm location, format, lint, test, stage, commit, push — in the correct order, and explain why each step exists.
4. Connect a local repository to a new GitHub repo and push for the first time.
5. Write a descriptive Git commit message, and read `git log --oneline` to verify a session’s commit history.

3 Before Class — Setup Checklist

- ☐ Rehearse the full ritual (Part 3) yourself, end to end, on your own demo machine before class — this is the one sequence in the entire course you want to be able to run without hesitation, since you’ll be repeating some version of it in front of the room essentially every week from here on.
- ☐ Confirm your own GitHub account and a scratch repo are ready to demonstrate Part 4’s remote-connection steps live — walk through creating a **Private** repo with **no README** (the lab page’s own specification) so students see exactly what “no README” avoids (a merge conflict on the very first push, from GitHub’s auto-created README colliding with the local repo’s own initial commit).
- ☐ Decide your policy on commit message quality in advance, and be ready to reject a vague one live if a student volunteers theirs for the room to see — “final” or “update” are explicitly called out by the lab page as unacceptable, and modeling that standard once, publicly, sets the tone for the rest of the semester’s grading.

4 Materials Needed

- Terminal (zsh), VS Code, Python 3.10+, ripgrep, pytest, ruff — from Week 3’s venv
- Students: their existing `ism3232/module02_zsh/` folder with `.venv/`, `.gitignore`, and `.zshrc` aliases from Week 3
- A GitHub account and internet access for Part 4

5 Timing Plan (75 minutes)

Time	Segment	Minutes
0:00–0:04	Welcome: “the most important lab of Unit 1”	4
0:04–0:16	Part 1 — Search tools	12
0:16–0:28	Part 2 — Write a test file	12
0:28–0:46	Part 3 — Pre-submission ritual	18
0:46–1:04	Part 4 — First GitHub push + verify commit log	18
1:04–1:10	Stretch preview (fzf/zoxide)	6
1:10–1:15	Wrap-up, submission checklist	5

Four required parts fill the class period; Part 3 and Part 4 are deliberately given the most time, since they’re both sequential, multi-step processes where a single skipped or misordered step causes real, confusing downstream failures — worth moving through carefully rather than quickly.

6 Segment-by-Segment Walkthrough

6.1 Intro (0:00–0:04)

Say to the class:

“This is the most important lab of Unit 1, and I want you to actually hear that, not just read it in passing. Today you learn a five-step ritual — location, format, lint, test, commit-and-push — that you will run, in some form, before nearly every submission for the rest of this course. I check the Git log. A missing ruff step or a missing pytest run is visible there, every time. Get this right today, once, carefully, and it becomes automatic.”

Do: Write the five ritual steps on the board — **Confirm** → **Format** → **Lint** → **Test** → **Commit/Push** — and leave it visible through Part 3.

6.2 Part 1 — Search Tools (0:04–0:16, 12 min)

Teaching goal: Three complementary search tools — `ripgrep` for searching file *contents*, `find` for locating files by *name*, `tree` for visualizing *structure* — each answering a genuinely different question.

Say to the class:

“Three tools, three different questions. `ripgrep` searches inside files for text. `find` searches for files by name. `tree` shows you the shape of a project. Confusing these three is the most common mistake — I want you leaving today knowing exactly which one to reach for.”

Live-code this, from inside `~/ism3232/`:

```
cd ~/ism3232
```

```
# ripgrep searches
rg 'print'           # every print statement in all Python files
rg 'def '            # every function definition
rg -l 'import'       # list filenames that contain 'import'

# find
find . -name '*.py'  # all Python files in the project tree
find . -name '*.md'  # all README files
find . -name '*.txt' # all text files

# tree
tree -L 3            # full project structure 3 levels deep
```

Line-by-line explanation:

- `rg 'print'` — **ripgrep** searches the *contents* of every file (respecting `.gitignore`, by default — worth mentioning as a genuinely nice built-in behavior: it won't waste time searching inside `.venv/`, since that's already excluded) for the literal text `print`, printing every matching line along with its filename and line number.
- `rg 'def '` — same tool, different search term — note the trailing space in `'def '`: this specifically matches function *definitions* (`def something(...)`), not every occurrence of the substring `def` anywhere (which could otherwise match inside unrelated words). A good, concrete illustration that a search pattern's exact text matters.
- `rg -l 'import'` — the `-l` flag changes ripgrep's output mode: instead of showing every matching *line*, it lists just the **filenames** that contain at least one match — useful when you want to know *which* files are relevant without being flooded by every individual line.
- `find . -name '*.py'` — **find** is a fundamentally different tool: it searches for files **by name pattern**, not by their contents. `.` means “starting from the current directory”; `-name '*.py'` matches any filename ending in `.py`, using `*` as a wildcard. Say explicitly, since this is the core distinction worth drilling: `find . -name '*.py'` cannot tell you what's *inside* those files — for that, you'd combine it with `rg`, or use `rg` alone in the first place if content is what you actually care about.
- `tree -L 3` — Week 2's `tree`, now at depth 3 instead of 2, appropriate for `ism3232/`'s deeper nested structure now that several modules have their own subfolders (`.venv/`, `tests/`, etc.).

Point out explicitly, comparing the three tools directly: “If I asked you to find every place in this project where the word `TODO` appears, which tool? [`rg 'TODO'`]. If I asked you to find a file but you only remembered it was named something like `report`? [`find . -name '*report*'`]. If I asked you to just get oriented in an unfamiliar project for the first time? [`tree`].” Getting the room to answer these three quickly, out loud, is a better check than any amount of further explanation.

Common student mistakes to watch for:

- Reaching for `find` when the actual goal is searching file *contents* — e.g., trying `find . -name '*print*'` to locate `print` statements, which searches *filenames*, not code, and correctly finds nothing (since no file is literally named “`print`”). A good, concrete “wrong tool for the question” moment worth letting play out rather than correcting immediately.
- Forgetting quotes around the search pattern (`rg def` instead of `rg 'def '`) — often still works for a single word with no special characters, but breaks (or behaves unexpectedly) the moment the pattern includes a space or special character, as `'def '` does here; worth establishing quoting as a default habit now rather than an afterthought later.
- Running these commands from the wrong directory (not `~/ism3232`) — produces a much smaller (or empty) result set, not an error; a good `pwd` check if a student's output looks suspiciously thin compared to their neighbor's.

Check for understanding: “If you needed to find every Python file that contains the word `TODO`, name the single ripgrep command that does both jobs at once, without a separate `find` step.” (`rg`

`-l 'TODO' --glob '*.py'`, or simply `rg 'TODO'` restricted implicitly to text files it can search — the point isn't precise flag syntax so much as recognizing that `rg` alone can often answer “which files have this content” without needing `find` at all, since content search and filename filtering can combine in one tool.)

6.3 Part 2 — Write a Test File (0:16–0:28, 12 min)

Teaching goal: Write a real pytest file with three test functions, and run it — a first, genuine exposure to automated testing, ahead of this course's Unit 2 programming modules.

Say to the class:

“Three tests, each checking something simple and true, so you can see what a passing test suite looks like before you ever have to debug a failing one.”

Live-code this:

```
cd ~/ism3232/module02_zsh
source .venv/bin/activate
mkdir -p tests
touch tests/__init__.py tests/test_week4.py
code tests/test_week4.py
```

Type this into the file:

```
# tests/test_week4.py

def test_always_passes():
    assert 2 + 2 == 4

def test_string_is_lowercase():
    name = 'ism3232'
    assert name == name.lower()

def test_path_segments():
    path = '/Users/yourname/ism3232/module02_zsh'
    parts = path.split('/')
    assert 'ism3232' in parts
```

Line-by-line explanation:

- `mkdir -p tests` — creates a `tests/` folder; `-p` (from Part 3's `mkcd` function last week) means “don't error if it already exists, and create any needed parent folders” — a safe, idempotent way to ensure the folder exists regardless of whether this is the first time running the command.
- `touch tests/__init__.py tests/test_week4.py` — two files at once; `__init__.py`, even empty, is a convention marking `tests/` as a proper Python package, which helps pytest's test discovery machinery — worth a one-sentence mention without a deep dive into Python packaging today.
- `def test_always_passes():` — **pytest automatically discovers any function whose**

name starts with test_ — this is a naming *convention*, not a decorator or special syntax; say explicitly that this is why the filename (`test_week4.py`) and every function inside it are prefixed `test_` — `pytest` is specifically looking for that pattern to know what to run.

- `assert 2 + 2 == 4` — **assert** is new syntax today: it checks that the following expression is `True`, and if it *isn't*, raises an error and the test is marked as **failed**. If the assertion holds, nothing visible happens — the test simply passes silently. Say explicitly: this is the entire mechanism of automated testing — write a statement that should be true if your code is correct, and let `pytest` tell you immediately if it isn't.
- `test_string_is_lowercase` — `'ism3232' == 'ism3232'.lower()` — since the string is already lowercase, `.lower()` doesn't change it, so this assertion holds. A good moment to ask: “what would happen to this test if name were `'ISM3232'` instead?” (It would still pass — `.lower()` would convert it to `'ism3232'`, matching the comparison — a good check that students are tracing the assertion's actual logic, not just its current input.)
- `test_path_segments` — `path.split('/')` breaks a path string into a list of its segments at every `/` character (a genuinely useful string method, worth naming explicitly since it's the first time this semester a string is programmatically decomposed this way); `'ism3232'` in parts checks whether that specific string is present anywhere in the resulting list — asserting that it is.

Run it:

```
pytest -v
```

Verified output:

```
tests/test_week4.py::test_always_passes PASSED
tests/test_week4.py::test_string_is_lowercase PASSED
tests/test_week4.py::test_path_segments PASSED
3 passed in 0.00s
```

Line-by-line explanation of the flag: `-v` means **verbose** — without it, `pytest` prints a much more compact summary (a single line of dots, one per test); `-v` instead names each test individually alongside its result, which is worth using while learning specifically because it makes the connection between a named test function and its pass/fail status immediately visible.

Common student mistakes to watch for:

- Naming a test function without the `test_` prefix (e.g., `check_always_passes`) — `pytest` simply won't discover or run it at all, silently — no error, just zero output for that function; if a student's test count doesn't match what they expect, checking function names against the naming convention is the first thing to verify.
- Using `==` inside the `assert` where a typo or logic error makes the comparison always false — walk through reading a failure's actual output together if this happens naturally, since a genuine, if accidental, failing test is a better first exposure to `pytest`'s failure-reporting format than any staged example.
- Running `pytest -v` **without** activating the `venv` first — if `pytest` isn't found at all (command

not found: pytest), that's the tell; a good direct callback to Week 3's "was the venv actually activated" troubleshooting habit.

Check for understanding: "If `test_path_segments` used `path.split('\\')` (a backslash) instead of `path.split('/')`, on this specific path string, would the test still pass?" (No — the path uses forward slashes; splitting on a backslash that never appears would return the whole string as one single "segment," and `'ism3232'` in parts would then check whether the string `'ism3232'` matches that one giant segment exactly, which it wouldn't — the assertion would fail. A good check that students are tracing what `.split()` actually does with the specific character given, not just pattern-matching the method name.)

6.4 Part 3 — Pre-Submission Ritual (0:28–0:46, 18 min)

Teaching goal: The single most important sequence in this entire course — confirm location, format, lint, test, then stage/commit/push, in that exact order, every time, for the rest of the semester.

Say to the class:

“This is the ritual. I am going to run it slowly, once, all the way through, narrating why each step exists — and then every one of you runs it yourselves, on your own machine, before we move to Part 4. This exact sequence, or something extremely close to it, is what you’ll run before every major submission this semester.”

Live-code the complete ritual, in order, without skipping or reordering any step:

```
# 1. Confirm location and structure
pwd
tree -L 3

# 2. Activate the virtual environment
source .venv/bin/activate

# 3. Format and lint
ruff format .
ruff check .

# 4. Run tests
pytest

# 5. Stage, commit, and push
git status
git add .
git commit -m 'lab 4: search ritual and git'
git push
```

Line-by-line explanation, one step at a time:

- **Step 1 — `pwd` then `tree -L 3`.** Confirm exactly where you are and what’s actually here, before doing anything — the same location-first discipline as Week 2’s `rm` safety ritual, now applied to an entire submission process instead of a single deletion.
- **Step 2 — `source .venv/bin/activate`.** Everything from here forward (`ruff`, `pytest`) needs to run inside the project’s isolated environment, not system-wide — Week 3’s habit, now load-bearing for the ritual to work at all.
- **Step 3 — `ruff format .` then `ruff check .`** Two related but distinct jobs: `ruff format .` automatically **rewrites** your code to a consistent style (spacing, quote style, line length) — say explicitly this actually *changes files on disk*, which is worth knowing before running it

on code you haven't saved a backup of, though for this course's lab work that's rarely a real concern. `ruff check .` then **lints** — scans for actual problems (unused imports, undefined names, common bug patterns) without changing anything, just reporting. The order matters: format first, so linting runs against already-clean formatting rather than flagging style issues alongside real problems.

- **Step 4 — pytest.** Note this is run **without** `-v` here, unlike Part 2 — the ritual's version is meant to be a fast pass/fail gate, not a detailed line-by-line report; if something fails, `-v` (or just reading the failure output directly) is the natural next step, but the ritual's default is the terser form.
- **Step 5 — `git status`, `git add .`, `git commit -m '...'`, `git push`.** Four sub-steps, each worth naming individually: `git status` shows what's changed since the last commit (nothing has been done to the repo yet at this point in this lab if it's the first-ever commit — that's fine, and Part 4 sets up the actual GitHub connection). `git add .` **stages** every changed file in the current directory and below. `git commit -m '...'` **records** a permanent snapshot, labeled with a message — say explicitly, echoing the lab page's own warning: **'lab 4: search ritual and git' is specific and descriptive; 'final' or 'update' are not, and this course's grading looks at commit message quality directly.** `git push` uploads the commit to GitHub — Part 4 sets up exactly where it goes for the very first time.

Ritual complete when, per the lab page's own explicit criteria — state this checklist out loud, and have every student confirm all three before moving on:

- `ruff check` returns **no errors**
- `pytest` returns **all passed**
- `git status`, run again after the push, says **“nothing to commit”**

Common student mistakes to watch for:

- Running the steps out of order — most commonly, committing/pushing *before* running `ruff/pytest`, which means a broken or unformatted submission goes out before it's actually verified. Say explicitly: the order isn't arbitrary — each step is a gate the next one assumes has already passed.
- `ruff check` reporting real errors and the student pushing anyway without fixing them — stop and actually read what `ruff` flagged together; this is a genuine “was the code actually checked” moment worth not skipping past under time pressure.
- A vague commit message, discovered only when reviewing `git log` later — catch this live, in Part 3, before the push happens, rather than after; it's much easier to fix a not-yet-pushed commit message (`git commit --amend -m '...'`, not covered in depth today but worth mentioning exists) than to leave a vague one in permanent history.

Check for understanding: “If `ruff check` reported an error, would you fix it before or after running `pytest`?” (Before — the ritual's stated order runs format/lint ahead of tests specifically so obvious code-quality issues are caught and resolved first, before spending time investigating whether a test failure is a real logic bug or just a symptom of messy, unchecked code. Getting a student to

articulate *why* the order matters, not just recite it, is the real check here.)

6.5 Part 4 — First GitHub Push (0:46–1:04, 18 min)

Teaching goal: Create a new GitHub repository and connect the local project to it for the first time — completing the loop the ritual’s final step (`git push`) assumed already existed.

Say to the class:

“Part 3’s `git push` only works once there’s actually somewhere to push to. If you haven’t connected this project to GitHub yet, here’s exactly how, one time, for this project.”

Do, live, in the browser first: Create a new repository on github.com — **name it `ism3232-module02`, set it Private, and do not initialize it with a README**. Say explicitly why “no README”: if GitHub creates a README automatically, and your local repo also already has its own initial commit, the two histories diverge immediately and your very first `git push` fails with a conflict — avoiding this now, on the very first repo of the semester, is worth the explicit callout.

Then, in the terminal:

```
git remote add origin https://github.com/YOURUSERNAME/ism3232-module02.git
git branch -M main
git push -u origin main
```

Line-by-line explanation:

- `git remote add origin https://github.com/...` — tells your **local** Git repository about a **remote** location (`origin` is just a conventional name for “the primary remote,” not a required keyword) where it can push to and pull from — say explicitly: up to this point, `git commit` has only ever created *local* history; this line is what makes `git push` have an actual destination.
- `git branch -M main` — renames the current branch to `main` (the modern default branch name convention), with `-M` forcing the rename even if a branch with that target name situation could otherwise conflict — worth a brief note that `main` (not the older `master` convention) is what this course, and most current GitHub repos, standardize on.
- `git push -u origin main` — pushes the `main` branch to the `origin` remote; `-u` (short for `--set-upstream`) links this local branch to that specific remote branch **permanently**, so every *future* `git push` from this project can just be `git push` alone, with no arguments — say explicitly this is a one-time setup step specifically *because* of the `-u` flag; without it, every future push would need the full `origin main` spelled out again.

Verify: open the GitHub repository in a browser and confirm the pushed files are visible.

Then, verify the commit log:

```
git log --oneline    # should show your commits
git status          # should say 'nothing to commit'
```

Say explicitly, restating the lab page’s own grading note: “The commit log is part of your grade, every week from here forward. `git log --oneline` should show real, descriptive messages —

not 'final', not 'update'. Instructors read this log; it's not a private implementation detail, it's a visible record of how carefully you worked."

Common student mistakes to watch for:

- Creating the GitHub repo **with** a README, despite the explicit instruction not to — if this happens, the resulting push conflict is a genuinely good, low-stakes teaching moment about *why* the instruction existed, rather than something to just quickly work around; walk through `git pull --rebase` or deleting and recreating the GitHub repo cleanly, whichever is faster for your specific class's time budget.
- Forgetting to replace YOURUSERNAME with their actual GitHub username in the remote URL — produces `remote: Repository not found`, a good, specific error worth reading together, since the fix is visually obvious once pointed at.
- Running `git push` (without `-u origin main`) on this very first push, before the upstream link is established — fails with a `fatal: The current branch main has no upstream branch` error that explicitly suggests the correct fix in its own output — worth having a student read that suggested fix aloud rather than you supplying it immediately.

Check for understanding: "After today's `-u` flag, what does a plain `git push` (no arguments) do on this specific project, next week and beyond?" (Pushes the current branch to the already-linked `origin main` automatically — get a student to state explicitly that this is *why* the ritual's Step 5 can just say `git push`, with no arguments, and it still works correctly every week after this one.)

6.6 Stretch — fzf or zoxide (1:04–1:10, as time allows)

Frame as a genuine bonus, worth showing even briefly since both tools are broadly useful beyond this course:

```
fzf          # interactive fuzzy file search
z ism3232    # jump to course folder from anywhere
z module02   # jump to module02_zsh
```

One sentence each, if you demo this live: “fzf opens an interactive, type-to-filter search over files in the current directory — genuinely faster than `find` once a project has many files and you only remember part of a name. `z` (from the `zoxide` tool) is a smarter `cd` — it learns which folders you visit most often and lets you jump to them by typing just part of the name, from *anywhere*, not just from a parent directory the way plain `cd` requires.” Neither is required for this course’s ritual, but both are the kind of tool that, once discovered, tends to stick with a working developer for years.

7 Wrap-Up (last ~5 minutes)

Review the submission checklist together, on screen:

- ☐ **Screenshot 1:** Two `rg` commands and `tree -L 3` output
- ☐ **Screenshot 2:** The complete pre-submission ritual — all steps in one terminal session
- ☐ **Screenshot 3:** GitHub repository page showing committed files
- ☐ **File upload:** `module02_zsh/tests/test_week4.py`
- ☐ **Also paste:** GitHub repository URL into the Canvas text box
- ☐ All uploaded to Canvas by end of class

Preview Week 5: “Everything through Week 4 has been about the environment and the workflow around code — search tools, testing infrastructure, the submission ritual, Git. Starting next week, the course turns to Python itself: variables, data types, and operators, the actual programming content this workflow exists to support.”

8 Appendix A — Full Command Reference

Part 1 (search tools, from `~/ism3232`):

```
rg 'print'
rg 'def '
rg -l 'import'
find . -name '*.py'
find . -name '*.md'
find . -name '*.txt'
tree -L 3
```

Part 2 (`tests/test_week4.py`):

```
cd ~/ism3232/module02_zsh
source .venv/bin/activate
mkdir -p tests
touch tests/__init__.py tests/test_week4.py
```

```
def test_always_passes():
    assert 2 + 2 == 4

def test_string_is_lowercase():
    name = 'ism3232'
    assert name == name.lower()

def test_path_segments():
    path = '/Users/yourname/ism3232/module02_zsh'
```

```
parts = path.split('/')
assert 'ism3232' in parts
```

```
pytest -v
```

Part 3 (the ritual — memorize this sequence):

```
pwd
tree -L 3
source .venv/bin/activate
ruff format .
ruff check .
pytest
git status
git add .
git commit -m 'lab 4: search ritual and git'
git push
```

Part 4 (first GitHub push):

```
git remote add origin https://github.com/YOURUSERNAME/ism3232-module02.git
git branch -M main
git push -u origin main
git log --oneline
git status
```

Stretch:

```
fzf
z ism3232
z module02
```

9 Appendix B — Extra Practice (only if the class finishes early)

Four required parts fill the full class period at a normal pace, especially given Part 4's real risk of GitHub authentication/setup friction. If a section moves unusually fast:

Extra — a fourth test, self-written. Have students add a `test_credit_hours` function to `test_week4.py`, asserting something true about a value from Week 1's `hello_ism3232.py` (e.g., `assert 3 * 2 == 6`, mirroring the `weekly_hours` calculation), then re-run the full ritual (format, lint, test, commit, push) end to end on this one additional change — genuinely good rehearsal of running the *entire* ritual for a small, realistic change, not just once as a demo.

Extra — a deliberately failing test. Have students temporarily add `def test_deliberately_wrong(): assert 1 == 2`, run `pytest -v`, and read the failure output carefully — what does a **failing** assertion's output look like, compared to Part 2's all-passing run? Have them remove it afterward and re-run to confirm a clean pass before finishing — good, low-stakes first exposure to reading a

test failure, which will happen for real, unplanned, later this semester.